

Normalización de Bases de Datos: Optimización de Sistemas Logísticos

Explicar el proceso de normalización de datos (1FN, 2FN y 3FN) a través de un ejemplo práctico de logística de e-commerce para mejorar la integridad y eficiencia de la información.

Estado Inicial (No Normalizado)

Cliente:	Juan Pérez
Dirección:	Av. Siempre Viva 123
Envío:	1001
Fecha:	2024-01-01
Paquetes:	PK101, PK102
Destino:	Madrid
Tarifa:	50.00
Rastreo:	987554321, 907554322

El Caos de la Tabla Única

En un sistema no normalizado, una sola tabla contiene toda la información mezclada; datos del cliente, detalles del envío y múltiples códigos de rastreo en una sola celda.

Celdas No Atómicas

La columna "Paquetes" contiene valores como "PK101, PK102", lo que imposibilita realizar búsquedas o filtros eficientes por paquete individual.

Primera Forma Normal (1FN): Atomicidad

Regla de Oro: Un Valor por Celda

La 1FN exige que cada celda contenga un único valor atómico y que exista una Clave Primaria (PK) única para identificar cada fila.

Eliminación de Grupos Repetitivos

Envío	Paquete
1001	PK101
1001	PK102

Se desglosan los códigos de rastreo; si un envío tiene dos paquetes, ahora existen dos filas distintas, permitiendo el control individual de cada unidad.

Segunda Forma Normal (2FN): Dependencias Parciales

Dependencia de la Clave Completa

Para estar en 2FN, la tabla debe estar en 1FN y todos los atributos deben depender de la clave primaria completa, no solo de una parte de ella.

Separación de Entidades: Cliente vs. Envío

Cientes
ID_Cliente
Nombre, Dirección

Envios
ID_Envio
ID_Cliente
ID_Ciudad
Fecha_Envio

Se elimina la redundancia creando una tabla de 'Clientes' separada de los 'Envios', evitando repetir los datos del cliente en cada paquete enviado.

Tercera Forma Normal (3FN): Dependencias Transitivas

Eliminación de Dependencias Indirectas

La 3FN requiere que los atributos no clave dependan exclusivamente de la clave primaria, eliminando dependencias entre columnas que no son claves.

Independencia de Tarifas y Destinos

Tarifas_Destino
ID_Ciudad
Nombre_Ciudad
Estado
Tarifa_Envio

Envios
ID_Envio
ID_Cliente
ID_Ciudad
Fecha_Envio

La información de la ciudad y su tarifa se mueve a una tabla independiente, ya que la tarifa depende de la ciudad, no directamente del ID de envío.

Implementación Técnica (SQL)

```
-- Almacenar datos básicos de clientes.
CREATE TABLE Clientes (
  ID_Cliente INT PRIMARY KEY,
  Nombre VARCHAR(100),
  Direccion TEXT);

-- Gestionar costos de envío por ubicación.
CREATE TABLE Tarifas_Destino (
  ID_Ciudad INT PRIMARY KEY,
  Nombre_Ciudad VARCHAR(100),
  Estado VARCHAR(50),
  Botada VARCHAR(10),
  Tarifa_Envio DECIMAL(10,2));

-- Vincular clientes con sus destinos.
CREATE TABLE Envios (
  ID_Envio INT PRIMARY KEY,
  ID_Cliente INT,
  ID_Ciudad INT,
  Fecha_Envio DATE,
  FOREIGN KEY (ID_Cliente) REFERENCES Clientes(ID_Cliente),
  FOREIGN KEY (ID_Ciudad) REFERENCES Tarifas_Destino(ID_Ciudad));

-- Registro atómico de cada paquete enviado.
CREATE TABLE Detalle_Paquetes (
  ID_Paquete VARCHAR(50) PRIMARY KEY,
  ID_Envio INT,
  Peso_KG DECIMAL(5,2),
  FOREIGN KEY (ID_Envio) REFERENCES Envios(ID_Envio));
```

Conclusión y Beneficios

Prevención de Anomalías Críticas

La normalización previene errores de inserción (no poder añadir una ciudad sin envíos), actualización (cambiar una dirección en 50 filas) y borrado (borrar al cliente al borrar su único envío).

Eficiencia en Sistemas Logísticos

Al reducir la redundancia, las bases de datos son más rápidas, ocupan menos espacio y mantienen una "única fuente de verdad" confiable.

Referencias Consultadas

Contenido basado en estándares de la industria documentados por freeCodeCamp y Boot.dev.